SUREBITE ARCHITECTURE REVIEW

Backend Integration & Architecture Specification

Comprehensive Technical Blueprint, API Contracts, JSON Schemas & Multer Guidelines for MERN Backend Engineering

Target Service: Express.js + MongoDB / Mongoose + JWT **Frontend:** React 18 + Vite + Tailwind CSS **Standard:** AllergyMenu v2.1 Compliance

1. The API Services & Endpoints Contract

Encapsulated in `frontend/src/api/api.js` with Axios. When `VITE_USE_MOCK=false`, targets `VITE_API_URL` (default `http://localhost:5000/api`). Manager routes append `Authorization: Bearer <token>` from `localStorage.getItem('surebite_token')`.

Domain	Method	Endpoint	Auth	Request Payload / Headers	Expected Response Shape
Auth	POST	`/auth/register`	Public	`{ name, email, password, restaurantName }`	`{ token, user: { _id, name, email, restaurantName, restaurantSlug } }`
Auth	POST	`/auth/login`	Public	`{ email, password }`	`{ token, user: { _id, name, email, restaurantName, restaurantSlug } }`
Auth	GET	`/auth/me`	Bearer	*None*	`{ user: { _id, name, email, restaurantName, restaurantSlug } }`
Menus	GET	`/menus`	Bearer	*None*	`{ menus: [ menu, ... ] }` *(manager's owned menus)*
Menus	POST	`/menus/upload`	Bearer	`multipart/form-data` • `image` (File: JPG/PNG/WebP, ≤10MB) • `name` (String), `currency` (INR, USD, etc.)	`{ menu }` *(status: 'draft', with extracted items)*
Menus	GET	`/menus/:id`	Bearer	*None*	`{ menu: { ... } }`
Menus	PUT	`/menus/:id`	Bearer	`{ name, currency, items: [ ... ] }`	`{ menu: { ... } }` *(full replacement of items)*
Menus	PATCH	`/menus/:id/publish`	Bearer	*None*	`{ menu }` *(status: 'published', slug: 10 chars, publishedAt: ISO)*
Menus	GET	`/menus/:id/qr`	Bearer	*None*	`{ qrPng, qrSvg, publicUrl }`
Menus	DELETE	`/menus/:id`	Bearer	*None*	`{ success: true }`
Diner	GET	`/diner/:slug`	**Public**	*None*	`{ menu }` *(with restaurantName, sanitized owner)*
Diner	POST	`/diner/:slug/scan`	**Public**	`{}` *(telemetry / scan ping)*	`{ success: true }`

Crucial Error Contract: Frontend error helper `getErrorMessage(error)` reads `error.response.data.message` or `error.response.data.error`. Always return error responses as: `res.status(statusCode).json({ message: "Detailed explanation" })`.

2. The Mock Database Logic & JSON Shapes

When `VITE_USE_MOCK=true`, mock logic in `frontend/src/api/mockApi.js` stores data in `localStorage` (key: `surebite_mock_db`). Real MongoDB schemas must strictly match:

A. User Schema (MongoDB / Mongoose)

```
{
  "_id": "64f1a2b3c4d5e6f7a8b9c0d1",
  "name": "Ananya Shah",
  "email": "ananya@saffrontable.in",
  "restaurantName": "Saffron Table",
  "restaurantSlug": "saffron-table-e43z"
}
```

* Never expose the hashed password in any API response.

B. Menu Schema

```
{
  "_id": "64f1a2b3c4d5e6f7a8b9c0d2",
  "owner": "64f1a2b3c4d5e6f7a8b9c0d1",
  "name": "Dinner menu",
  "currency": "INR",
  "status": "published",   /* draft | published | failed */
  "slug": "e43ZvleZ1z",      /* 10 alphanumeric chars */
  "imageUrl": "/uploads/menu.jpg",
  "items": [ ... ],
  "createdAt": "2026-10-14T13:51:30.000Z",
  "updatedAt": "2026-10-14T13:55:00.000Z",
  "publishedAt": "2026-10-14T13:55:00.000Z"
}
```

C. Dish Item Schema (Subdocuments in `menu.items`)

```
{
  "_id": "64f1a2b3c4d5e6f7a8b9c0d3",
  "name": "Paneer tikka",
  "description": "Paneer, yogurt, bell pepper and cumin.",
  "price": 320,
  "category": "Starters",
  "dietaryType": "veg",          /* 'veg' | 'non_veg' | 'egg' */
  "allergens": ["dairy"],        /* array from the 14 canonical allergens */
  "aiInferred": false,           /* true if inferred by AI OCR, false if edited */
  "isVerified": true             /* must be manually confirmed by manager */
}
```

- **Manual dishes:** Sent in `PUT /menus/:id` without an `_id`. Mongoose must auto-generate `_id` for each dish.
- **Verification reset:** Any edits to dish title, description, category, or allergens resets `isVerified: false` in frontend.

3. Constants, Enums & Frontend Route Map

Defined in `frontend/src/utils/constants.js`. The backend must enforce validation matching these enums:

14 Canonical Allergens (EU/UK/FSSAI Standard):

`dairy` `gluten` `eggs` `peanuts` `tree_nuts` `soy` `fish` `shellfish` `sesame` `mustard` `celery` `lupin` `molluscs` `sulfites`

Dietary Types: `'veg'`, `'non_veg'`, `'egg'` • **Menu Statuses:** `'processing'`, `'draft'`, `'published'`, `'failed'` • **Currencies:** `'INR'` (₹), `'USD'` ($), `'EUR'` (€), `'GBP'` (£)

Frontend Route	Component	Access	Functional Purpose
`/`	Home.jsx	Public	Marketing landing page
`/auth`	Auth.jsx	Public	Manager sign in & restaurant workspace registration
`/dashboard`	Dashboard.jsx	Manager	Manage menus, review verification progress, delete menus
`/upload`	Upload.jsx	Manager	Photo upload & trigger for AI OCR extraction
`/menus/:id/verify`	Verify.jsx	Manager	Human verification: compare photo side-by-side, verify dishes & publish
`/menus/:id/qr`	QrCode.jsx	Manager	QR code download (PNG/SVG) and printable standing table tent
`/m/:slug`	DinerMenu.jsx	**Public (Diner)**	Interactive guest menu with real-time allergen & dietary filtering

4. The Upload Component & Multer Configuration

In `frontend/src/pages/Upload.jsx` (lines 114–118), the form is submitted via multipart `FormData`:

```
const formData = new FormData()
formData.append('image', file)          /* Field name: 'image' */
formData.append('name', name.trim())     /* Text field: Menu name */
formData.append('currency', currency)    /* Text field: INR, USD, EUR, etc. */
const { menu } = await api.menus.upload(formData)
```

Express + Multer Route Configuration

```
const multer = require('multer');
const path = require('path');

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 'uploads/'),
  filename: (req, file, cb) => {
    const unique = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, 'menu-' + unique + path.extname(file.originalname));
  }
});

const upload = multer({
  storage,
  limits: { fileSize: 10 * 1024 * 1024 }, /* 10 MB Max Limit */
  fileFilter: (req, file, cb) => {
    const allowed = ['image/jpeg', 'image/png', 'image/webp'];
    if (allowed.includes(file.mimetype)) cb(null, true);
    else cb(new Error('Choose a JPG, PNG or WebP image.'));
  }
});

router.post('/menus/upload', authenticateToken, upload.single('image'), menuController.uploadAndExtract);
```

5. Essential Backend Engineering Requirements

A. Publish Gatekeeper (Safety Rule)

> **Strict Safety:** `PATCH /menus/:id/publish` **MUST** reject if `items.length === 0` or if ANY dish has `isVerified === false` with `HTTP 400`: `{"message":"Every dish must be verified before the menu can be published."}`

- Set `menu.status = 'published'`
- Generate unique 10-char alphanumeric `slug` (e.g. `e43ZvleZ1z`)
- Set `menu.publishedAt = new Date().toISOString()`

B. Public Diner Menu Rules

Route: `GET /diner/:slug` (Public, no Bearer token)

- **Guard:** Only return menus with `status: 'published'`. Unpublished menus return `HTTP 404`: `{"message":"This menu is not available."}`.
- **Sanitization:** Inject `restaurantName` from owner into `menu.restaurantName`. **Never leak** the owner's `_id` or credentials.

C. QR Code Generation (`GET /menus/:id/qr`)

Reject with `HTTP 400` if menu is not published. Expected response:

```
{
  "publicUrl": "http://localhost:5173/m/e43ZvleZ1z",
  "qrPng": "...",
  "qrSvg": "<svg ...></svg>"
}
```

* Use NPM `qrcode`: `toDataURL(url, { width: 1024, margin: 2 })` and `toString(url, { type: 'svg', margin: 2 })`.

D. CORS & Environment Setup

```
/* app.js CORS */
app.use(cors({
  origin: process.env.CLIENT_URL || 'http://localhost:5173',
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization']
}));

/* backend/.env */
PORT=5000
MONGO_URI=mongodb://localhost:27017/surebite
JWT_SECRET=your_jwt_super_secret_key
CLIENT_URL=http://localhost:5173
GEMINI_API_KEY=your_gemini_vision_key
```